

Projeto 1

Simulador do modelo OSI em uma rede com múltiplos roteadores

Comunicação de Dados · Prof. Vinícius S. Borges

Orientações. O projeto consiste em um programa em Python que simula a arquitetura em camadas estudada na Aula 2 e exibe, passo a passo, o percurso de uma mensagem entre dois computadores separados por roteadores. Todo o conteúdo necessário está na Aula 2 e no Capítulo 2 de Forouzan. A topologia, os endereços e os casos de teste descritos aqui são obrigatórios e idênticos para todos: isso torna as execuções comparáveis entre si e comparáveis com o material interativo da aula. A escolha da biblioteca de interface é livre, desde que os requisitos de visualização da Seção 5 sejam atendidos. O programa deve rodar como executável de Windows, aberto com um duplo clique em máquina sem Python instalado.

Objetivos

Ao concluir o projeto, você deve ser capaz de:

- traduzir a descrição funcional de cada uma das sete camadas do modelo OSI em uma interface de software com responsabilidade única;
- implementar o **encapsulamento** e o **desencapsulamento**, reconhecendo a unidade de dados (*PDU*, unidade de dados de protocolo) própria de cada camada;
- distinguir na prática a entrega **salto a salto**, atribuída à camada de enlace, da entrega **origem ao destino**, atribuída à camada de rede, e da entrega **processo a processo**, atribuída à camada de transporte;
- operar os quatro níveis de endereçamento e observar quais deles mudam ao longo do percurso e quais permanecem fixos;
- quantificar o **custo do empilhamento**, isto é, a fração da transmissão ocupada por cabeçalhos.

Topologia de referência

A rede simulada tem três redes locais, quatro roteadores e cinco computadores, conforme a Figura 1. Os números sobre os enlaces são custos de encaminhamento e devem ser usados pelo algoritmo de escolha de rota.

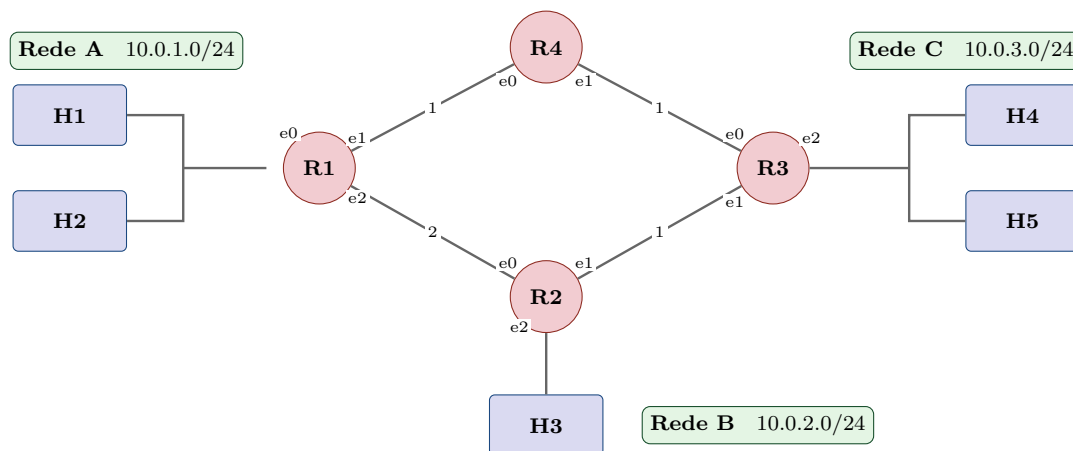


Figura 1: Topologia obrigatória. Os custos indicados sobre os enlaces tornam determinística a escolha de rota.

Os endereços lógicos e físicos são fixos e estão na Tabela 1. Os endereços físicos usam um formato reduzido de seis octetos, com o prefixo **AA** para computadores e **BB** para roteadores, apenas para facilitar a leitura no registro de eventos.

Dispositivo	Interface	Endereço lógico	Endereço físico
H1	eth0	10.0.1.10	AA:00:00:00:01:0A
H2	eth0	10.0.1.11	AA:00:00:00:01:0B
H3	eth0	10.0.2.10	AA:00:00:00:02:0A
H4	eth0	10.0.3.10	AA:00:00:00:03:0A
H5	eth0	10.0.3.11	AA:00:00:00:03:0B
R1	e0 (Rede A)	10.0.1.1	BB:00:00:00:01:00
	e1 (para R4)	10.0.14.1	BB:00:00:00:01:01
	e2 (para R2)	10.0.12.1	BB:00:00:00:01:02
R2	e0 (para R1)	10.0.12.2	BB:00:00:00:02:00
	e1 (para R3)	10.0.23.2	BB:00:00:00:02:01
	e2 (Rede B)	10.0.2.1	BB:00:00:00:02:02
R3	e0 (para R4)	10.0.34.3	BB:00:00:00:03:00
	e1 (para R2)	10.0.23.3	BB:00:00:00:03:01
	e2 (Rede C)	10.0.3.1	BB:00:00:00:03:02
R4	e0 (para R1)	10.0.14.4	BB:00:00:00:04:00
	e1 (para R3)	10.0.34.4	BB:00:00:00:04:01

Tabela 1: Endereçamento da rede simulada.

A topologia, os endereços e os custos devem ser lidos de um arquivo externo (por exemplo, `topologia.json`), e não escritos no meio do código. Um mesmo programa deve conseguir simular outra rede apenas com a troca desse arquivo, colocado ao lado do executável. Nenhum caminho absoluto pode aparecer no código.

O que cada camada deve fazer

Cada dispositivo possui uma pilha de camadas. Computadores implementam as sete camadas; roteadores implementam apenas as três primeiras, como visto na aula. Cada camada deve ser uma classe separada, com dois métodos de sentido oposto: um que recebe a unidade de dados da camada superior e a entrega à inferior, e outro que faz o caminho inverso. Nenhuma camada pode acessar diretamente uma camada que não seja adjacente a ela.

Restrições de projeto

As cinco restrições abaixo separam uma implementação correta de uma implementação que apenas parece correta. Todas são obrigatórias.

- R1. O roteador não lê o que não lhe pertence.** O objeto que representa o roteador não pode acessar campos das camadas 4 a 7. Se o código de um roteador precisar consultar uma porta ou o nome do processo, há um erro de modelagem.
- R2. O quadro não é reescrito.** A cada salto, o quadro recebido é descartado após a extração do pacote, e um quadro **novo** é construído na saída. O simulador deve exibir quadros distintos e numerados em cada enlace.
- R3. Endereços lógicos são fixos, endereços físicos são locais.** O par de endereços lógicos inserido na origem permanece o mesmo até o destino. O par de endereços físicos é substituído em cada salto.
- R4. A camada de enlace não escolhe rota.** A decisão de encaminhamento pertence exclusivamente à **camada 3**. A camada 2 apenas entrega ao vizinho que a camada 3 indicou.

N	Camada	Responsabilidade mínima exigida	Unidade
7	Aplicação	Gerar a mensagem e identificar o processo pelo nome (endereço específico).	Mensagem
6	Apresentação	Converter texto em sequência de octetos, registrar o esquema de codificação e cifrar o conteúdo, que só é decifrado na camada 6 do destino.	Mensagem
5	Sessão	Abrir, manter e encerrar o diálogo, atribuindo um identificador de sessão.	Mensagem
4	Transporte	Numerar portas de origem e destino, segmentar a mensagem em pelo menos três segmentos numerados quando ela exceder o limite adotado, e remontá-los em ordem na camada 4 do destino.	Segmento
3	Rede	Inserir o par de endereços lógicos e consultar a tabela de encaminhamento.	Pacote
2	Enlace	Inserir o par de endereços físicos do salto, delimitar o quadro e calcular a verificação de erro.	Quadro
1	Física	Converter o quadro em uma sequência de bits e transportá-la pelo enlace.	Bit

Tabela 2: Escopo mínimo de cada camada. Implementações mais completas são bem-vindas, desde que a responsabilidade de cada camada não invada a das demais.

R5. Cada camada conversa com a sua par. O cabeçalho inserido por uma camada só pode ser removido e interpretado pela camada de mesmo número no outro extremo do escopo correspondente: o vizinho para a camada 2, o destino final para as camadas 3 a 7.

O percurso que o simulador deve mostrar

A Figura 2 representa o caso central do projeto: uma mensagem de H1 para H4. Ela atravessa integralmente as sete camadas apenas nos extremos; nos roteadores sobe até a camada 3 e desce novamente. São quatro enlaces e, portanto, quatro quadros independentes, enquanto o pacote é um só.

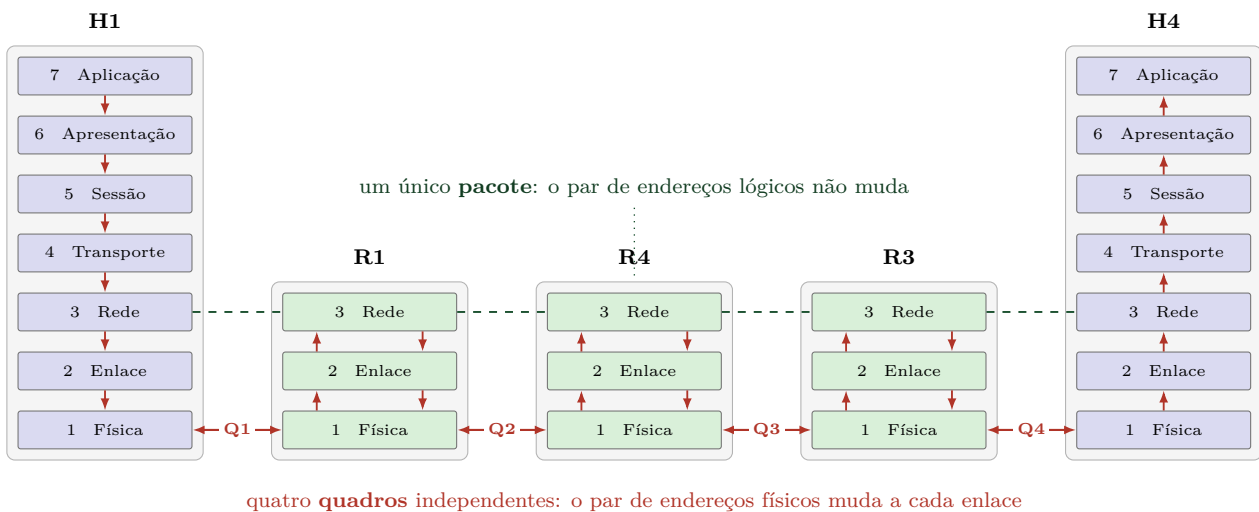


Figura 2: Percurso de H1 até H4 pelo caminho de menor custo. Os roteadores sobem apenas até a camada 3, onde a rota é decidida.

Requisitos de visualização

A biblioteca gráfica é de escolha livre. São aceitas, entre outras, `tkinter`, `pygame`, uma página servida localmente ou uma interface de terminal com `curses` ou `rich`. Independentemente da escolha, os sete requisitos abaixo devem estar presentes.

- V1. Mapa da rede.** Todos os dispositivos, enlaces e interfaces visíveis simultaneamente, com os nomes da Figura 1. O caminho efetivamente percorrido pela mensagem deve ser destacado.
- V2. Pilhas visíveis.** A pilha de camadas de cada dispositivo envolvido na comunicação deve estar na tela, com a camada ativa em destaque no instante em que ela processa a unidade de dados. Nos roteadores, a camada 3 deve destacar-se de modo evidente no momento da decisão de rota.
- V3. Unidade de dados desenhada.** A cada passo, o programa exibe a unidade de dados corrente como uma sequência de blocos, com os cabeçalhos já acrescentados à esquerda dos dados e o finalizador da camada 2 à direita. O nome da unidade acompanha a figura.
- V4. Os dois pares de endereços.** A tela deve mostrar, ao mesmo tempo, o par de endereços lógicos e o par de endereços físicos vigentes. Deve ficar visualmente claro que o primeiro permanece constante e o segundo é substituído a cada salto.
- V5. Controle de execução.** Três modos: passo a passo, execução contínua e pausa, com pelo menos três velocidades de reprodução.
- V6. Registro de eventos.** Uma área de texto rolável, em que cada linha corresponde a uma ação de uma camada de um dispositivo, no formato da Seção 5.1. O registro completo deve poder ser salvo em arquivo.
- V7. Alternância entre pilhas.** Um controle que troca a exibição entre a pilha OSI de sete camadas e a pilha TCP/IP, agrupando as camadas 5 a 7 em uma única camada de aplicação. A simulação subjacente não se altera: muda apenas o agrupamento mostrado na tela.

Formato do registro de eventos

Cada linha tem cinco campos separados por barra vertical: passo, dispositivo, camada, ação e descrição. O tamanho corrente da unidade de dados, em octetos, encerra a linha. As primeiras linhas do caso central devem ser semelhantes a estas:

001		H1		L7		GERA		processo navegador, destino servidorWeb	42 B
002		H1		L6		CODIFICA		octetos UTF-8, conteúdo cifrado	42 B
003		H1		L5		ABRE		sessão S-0001 estabelecida	46 B
004		H1		L4		SEGMENTA		porta 5210 → 443, segmento 1 de 1	54 B
005		H1		L3		ENCAPSULA		10.0.1.10 → 10.0.3.10	74 B
006		H1		L3		ROTEIA		próximo salto 10.0.1.1 pela interface eth0	74 B
007		H1		L2		ENQUADRA		AA:...:01:0A → BB:...:01:00, quadro Q1	92 B
008		H1		L1		TRANSMITE		736 bits no enlace H1-R1	92 B
009		R1		L1		RECEBE		736 bits do enlace H1-R1	92 B
010		R1		L2		DESENQUADRA		verificação de erro correta, quadro Q1 descartado	74 B
011		R1		L3		ROTEIA		10.0.3.0/24 via R4, custo 2, interface e1	74 B
012		R1		L2		ENQUADRA		BB:...:01:01 → BB:...:04:00, quadro Q2	92 B

Observe as linhas 010 e 012: o quadro Q1 é descartado e o quadro Q2 é construído, o que atende à restrição R2. Observe também que o endereço lógico de destino nunca reaparece alterado.

Casos obrigatórios

O programa deve executar os sete casos abaixo, selecionáveis pela interface. Cada um exercita um ponto distinto da Aula 2.

- C1. Entrega direta.** H1 envia para H2. Ambos estão na Rede A. A camada 3 deve reconhecer que o destino compartilha o prefixo de rede e entregar sem roteador. Há um único quadro.

- C2. Entrega indireta, caso central.** O processo `navegador`, porta 5210, em H1, envia para o processo `servidorWeb`, porta 443, em H4. Três roteadores, quatro quadros, um pacote.
- C3. Demultiplexação.** H1 e H2 enviam simultaneamente para o mesmo `servidorWeb` em H4, com portas de origem distintas. O simulador deve mostrar a camada 4 de H4 separando os dois fluxos e entregando cada um à sessão correta.
- C4. Falha de enlace.** O enlace R1–R4 é derrubado pela interface. A mensagem de H1 para H4 deve passar a seguir por R2, com custo total 3, e o mapa deve refletir o novo caminho.
- C5. Destino inalcançável.** H1 envia para 10.0.9.10, endereço que não pertence a nenhuma rede da topologia. O pacote deve ser descartado na camada 3 do primeiro roteador que não encontrar rota, com registro explícito do descarte.
- C6. Erro de transmissão.** Durante o caso C2, um bit do quadro é alterado no enlace R4–R3. A camada 2 do receptor deve detectar a inconsistência pela verificação de erro, descartar o quadro e registrar o descarte, sem que nenhuma camada superior seja acionada. A injeção do erro é acionada pela interface.
- C7. Mensagem longa.** H1 envia para H4 uma mensagem que excede o limite adotado na camada 4 e é dividida em pelo menos três segmentos. Cada segmento percorre a rede por conta própria, e a camada 4 de H4 só entrega a mensagem à camada 5 depois de remontar todos eles na ordem correta.

Custo do empilhamento

Ao final de cada execução, o programa deve apresentar o quadro numérico da comunicação: o tamanho útil da mensagem, o total de octetos efetivamente transmitidos somando todos os enlaces, e a eficiência

$$\eta = \frac{\text{octetos de dados}}{\text{octetos transmitidos}}, \quad \text{sobrecarga} = 1 - \eta.$$

Adote cabeçalhos de tamanho fixo: 4 octetos para a camada 5, 8 para a camada 4, 20 para a camada 3, 14 para o cabeçalho da camada 2 e 4 para o seu finalizador. Compare a eficiência do caso C1, de um único enlace, com a do caso C2, de quatro enlaces, e apresente os dois valores na tela.

Organização do código

A divisão em módulos abaixo é uma sugestão, não uma imposição. Ela é apresentada porque reproduz, no código, a mesma separação de responsabilidades que o modelo OSI propõe para a rede.

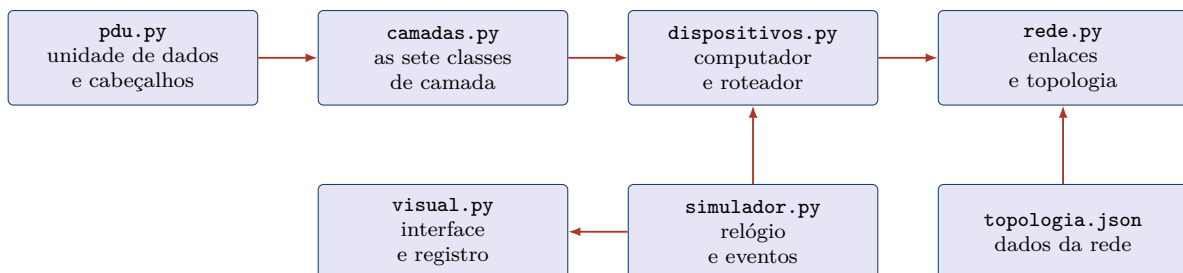


Figura 3: Separação sugerida em módulos. A interface gráfica consome eventos produzidos pelo simulador e não conhece a implementação das camadas.

Duas recomendações que evitam a maior parte dos problemas observados em projetos anteriores. A primeira: faça o simulador funcionar por completo em modo textual antes de escrever qualquer linha de interface gráfica. A segunda: a interface deve apenas **ler** uma lista de eventos produzida pelo núcleo da simulação; se a interface precisar chamar métodos das camadas para desenhar a tela, o acoplamento está errado.

Referência

FOROUZAN, B. A. *Comunicação de Dados e Redes de Computadores*. 6. ed. Porto Alegre: AMGH, 2022. Capítulo 2.